CustomScriptTutorial_2.py

コード構成・各処理 解説

ACP カスタムスクリプトの構成要素と、各設定・処理の役割を解説します。

Plant 3D ACP | デコレータ設定 | 形状生成（BOX） | ブーリアン演算 | 2層構造パターン

0 ファイル全体

以下がスクリプトファイルの全体像です。大きく **① インポート**・**② デコレータ（登録設定）**・**③ シェル関数**・**④ 形状生成関数（run）** の 4 つのブロックで構成されています。

① インポート → ② デコレータ（登録設定） → ③ シェル関数 → ④ run()（形状生成）

```python
# ── ① インポート ──────────────────────────────────────────────
from varmain.primitiv import *    # BOX / CYLINDER / CONE など形状プリミティブ
from varmain.custom  import *    # @activate / @param / @group デコレータ

# ── ② デコレータ（Plant 3D への登録設定） ────────────────────────
@activate(Group="CustomScriptTutorial", TooltipShort="Support",
          TooltipLong="Support CustomScriptTutorial",
          LengthUnit="mm", Ports="1")
@group("MainDimensions")
@param(A=LENGTH, TooltipShort="A", TooltipLong="A")

# ── ③ シェル関数（ACP インターフェース） ────────────────────────
def CustomScriptTutorial_2(s, A=100., **kw):
    return run(s, A=A, **kw)


# ── ④ 形状生成関数 ────────────────────────────────────────────
def run(s, A=100., **kw):
    main = BOX(s, L=A, W=100, H=100).translate((0, 0, 100 / 2))

    sub = BOX(s, L=50, W=150, H=50).translate((0, 0, -150 / 2))
    main.uniteWith(sub)
    sub.erase()

    sub = BOX(s, L=300, W=150, H=300).translate((0, 0, 150 / 2)).rotateX(45)
    main.subtractFrom(sub)
    sub.erase()

    return main
```

1 インポート

```
from varmain.primitiv import *
from varmain.custom  import *
```

Plant 3D ACP スクリプトに必要な 2 つのモジュールをすべて読み込みます。

モジュール	提供するもの
`varmain.primitiv`	形状プリミティブ関数 — `BOX`、`CYLINDER`、`CONE`、`SPHERE` など。3D ソリッドを生成するために使用する。
`varmain.custom`	ACP 登録用デコレータ — `@activate`、`@param`、`@group`。これらが Plant 3D への部品情報の登録を担う。

ポイント：両モジュールとも `import *` で読み込むため、スクリプト内で `BOX(...)` や `@activate` を修飾なしで使用できます。

2 デコレータ（Plant 3D への登録設定）

デコレータは **直後の関数に付加するメタ情報** です。ACP では 3 種類のデコレータを重ねて使用し、Plant 3D がこのスクリプトをカタログ部品として認識するための情報を定義します。

Plant 3D 側の処理： `PLANTREGISTERCUSTOMSCRIPTS` コマンドを実行すると、これらのデコレータの内容が読み取られ、XML 設定ファイル群（.xml / variants.map / ScriptGroup.xml）が**自動生成**されます。

@activate — カタログ登録情報

```
@activate(Group="CustomScriptTutorial",
          TooltipShort="Support",
          TooltipLong="Support CustomScriptTutorial",
          LengthUnit="mm",
          Ports="1")
```

引数	設定値	役割
`Group`	`"CustomScriptTutorial"`	Catalog Builder でのグループ分類名。同じ Group 名のスクリプトは同一グループとして扱われる。
`TooltipShort`	`"Support"`	部品選択時に表示される短い説明文。
`TooltipLong`	`"Support CustomScriptTutorial"`	部品選択時に表示される長い説明文（詳細情報欄）。
`LengthUnit`	`"mm"`	パラメータの長さ単位。`run()` 内の数値はすべてこの単位で解釈される。
`Ports`	`"1"`	配管接続口（ポート）の数。パイプへの接続点の個数を定義する。

@group — パラメータグループ

```
@group("MainDimensions")
```

直後に定義するパラメータを `"MainDimensions"` グループにまとめます。Plant 3D のプロパティパレットでグループ名として表示され、複数のパラメータを論理的にまとめるために使用します。

複数のパラメータグループを作る場合は `@group` を複数回使用します。`@group("寸法")` → `@param(A=LENGTH, ...)` → `@group("素材")` → `@param(Material=STRING, ...)` のように記述します。

@param — パラメータ定義

```
@param(A=LENGTH, TooltipShort="A", TooltipLong="A")
```

引数	設定値	役割
`A`	`LENGTH`	パラメータ名 `A` を `LENGTH`（長さ）型として定義。Plant 3D のプロパティパレットに入力欄として表示される。型には `LENGTH`、`ANGLE`、`INTEGER` などがある。
`TooltipShort`	`"A"`	パラメータの短い説明。プロパティパレットのラベルとして表示される。
`TooltipLong`	`"A"`	パラメータの詳細説明文。

注意：デコレータを変更した場合（パラメータの追加・削除・型変更）は、`PLANTREGISTERCUSTOMSCRIPTS` を再実行して XML を再生成し、**Plant 3D を再起動**する必要があります。

3 シェル関数（ACP インターフェース）

```
def CustomScriptTutorial_2(s, A=100., **kw):
    return run(s, A=A, **kw)
```

Plant 3D ACP が直接呼び出す関数です。**関数名がそのままスクリプト識別子**になるため、Catalog Builder の「使用する Python スクリプト形状」欄に入力する名前と一致している必要があります。

要素	役割
関数名 `CustomScriptTutorial_2`	ACP がこの名前でスクリプトを識別する。Catalog Builder で指定するスクリプト名と完全一致が必要。
引数 `s`	ACP が渡す形状生成コンテキスト（セッションオブジェクト）。`BOX(s, ...)` 等へそのまま渡す。直接操作することはない。
引数 `A=100.`	`@param` で定義したパラメータ。デフォルト値 `100.0`（mm）。プロパティパレットで値を変更すると、この引数として渡される。
`**kw`	ACP が追加で渡す可能性のある予備引数をまとめて受け取る。基本的には `run()` へそのままスルーする。
`return run(...)`	形状生成処理を `run()` に委譲する。ACP は戻り値の形状オブジェクトを 3D 図面に配置する。

2層構造の利点：このシェル関数はデコレータとインターフェースのみを担い、形状ロジックは `run()` に分離しています。これにより、`run()` の内容だけを Hot Reload 環境（hotreload_impl.py）でリアルタイム確認しながら開発し、完成後に `run()` をそのままコピペで本番ファイルに移行できます。

4 形状生成関数 run()

実際の 3D 形状を生成する処理本体です。プリミティブ形状（BOX）を組み合わせたブーリアン演算で最終形状を作ります。

```
def run(s, A=100., **kw):
    main = BOX(s, L=A, W=100, H=100).translate((0, 0, 100 / 2))

    sub = BOX(s, L=50, W=150, H=50).translate((0, 0, -150 / 2))
    main.uniteWith(sub)
    sub.erase()

    sub = BOX(s, L=300, W=150, H=300).translate((0, 0, 150 / 2)).rotateX(45)
    main.subtractFrom(sub)
    sub.erase()

    return main
```

BOX — プリミティブ形状の生成

```
main = BOX(s, L=A, W=100, H=100)
```

直方体ソリッドを生成します。原点を中心（底面中央）として配置されます。

引数	意味
`s`	コンテキスト（必須・そのまま渡す）
`L`	X 方向の長さ。ここではパラメータ `A` を使用
`W`	Y 方向の幅。100 mm 固定
`H`	Z 方向の高さ。100 mm 固定

translate() — 位置移動

```
.translate((0, 0, 100 / 2))
```

形状を (X, Y, Z) 方向に移動します。引数はタプル `(dx, dy, dz)` 。

この例では **Z 方向に 50 mm** 移動しています。BOX の底面が Z=0（原点）に来るよう、高さの半分だけ持ち上げています。

> BOX は原点を **底面中央** として生成されるため、`H/2` だけ Z 移動すると底面が Z=0 に揃います。

rotateX() — X 軸回転

```
.rotateX(45)
```

形状を **X 軸まわりに指定角度（度）だけ回転**します。

この例では切り取り用の BOX を 45° 傾けることで、斜めカットの効果を得ています。メソッドチェーンで `.translate(...).rotateX(45)` のように連続して使用できます。

uniteWith() — ブーリアン結合

```
sub = BOX(s, ...).translate(...)
main.uniteWith(sub)
sub.erase()
```

`main` と `sub` を **合体（和集合）**させます。

処理後、`main` が結合後の形状を保持します。`sub` 自体はソリッドとして残ったままなので、`sub.erase()` で明示的に削除します。

erase() を忘れずに：削除しないと図面に余分な形状が残ります。

subtractFrom() — ブーリアン切り取り

```
sub = BOX(s, ...).translate(...).rotateX(45)
main.subtractFrom(sub)
sub.erase()
```

`main` から `sub` の形状を **くり抜き（差集合）**ます。

この例では 45° 傾けた大きな BOX を使って `main` の一部を斜めにカットしています。`uniteWith` と同様、処理後は `sub.erase()` で削除します。

処理の流れまとめ

① **main BOX 生成** L=A, W=100, H=100 → ② **sub BOX を結合** uniteWith（底部拡張） → ③ **大 BOX を 45° 傾ける** rotateX(45) → ④ **斜めカット** subtractFrom → ⑤ **main を返す** return main

return main： `run()` は **最終形状オブジェクト 1 つ**を返す必要があります。ACP はこの戻り値を 3D 図面に配置します。複数のソリッドを返すことはできないため、ブーリアン演算でひとつの形状に統合してから返します。

5 スクリプトを変更するときのポイント

変更内容	再起動の要否	注意事項
`run()` 内の形状ロジックを変更する（BOX のサイズ・位置・ブーリアン演算など）	不要 Hot Reload 環境	hotreload_impl.py を編集して保存し、SYNCWRAPPER を実行するだけで反映される。
`@param` のパラメータを追加・削除・型変更する	必要	`PLANTREGISTERCUSTOMSCRIPTS` → XML 再生成 → Plant 3D 再起動が必要。
`@activate` の Group・Ports・LengthUnit を変更する	必要	同上。XML が再生成されるまで変更は反映されない。
関数名（スクリプト識別子）を変更する	必要	Catalog Builder の「使用する Python スクリプト形状」欄の名前も合わせて更新が必要。